

DevOps Interview Q&A Guide

AWS • Docker • Lambda • Security • Jenkins • Terraform • Deployments

Q1. What are the different ways to host Docker containers on AWS?

Lead with EKS since that's your hands-on experience, then mention others:

- EKS (Elastic Kubernetes Service) — Kubernetes-based; used in ApnaKart project
- ECS (Elastic Container Service) — managed orchestration, runs on EC2 or Fargate
- AWS Fargate — serverless compute for containers; no EC2 management
- AWS Lambda — container image support (up to 10 GB image)
- AWS App Runner — fully managed, auto-scales, good for web apps
- EC2 directly — install Docker on EC2, run containers manually (least preferred)
- AWS Batch — for batch workloads in containers

Q2. Have you ever deployed a Docker container on AWS Lambda?

You haven't done this directly — don't fake it. Honest answer:

"I haven't deployed a container on Lambda personally. In my project, I used EKS for containerized workloads. But I understand the concept — Lambda supports container images up to 10 GB packaged via ECR. You push the image to ECR, then configure the Lambda function to use that image URI instead of a zip-based deployment. The container must implement the Lambda Runtime API."

Q3. What is the maximum container image size for Lambda?

10 GB — this is the hard limit for container images on Lambda.

Q4. What is the maximum memory for Lambda?

10,240 MB (10 GB) — Lambda scales CPU proportionally with memory allocation.

Q5. What is a Layer in Lambda?

A Layer is a zip archive that contains libraries, dependencies, or custom runtimes that you want to share across multiple Lambda functions — without bundling them into every deployment package.

Example: If 5 Lambda functions all use pandas and numpy, you create one Layer with those packages and attach it to all 5. Reduces duplication and keeps function packages small.

- Max 5 layers per function
- Total unzipped size (function + layers) must be 250 MB or less

Q6. What kind of security scans do you run and on what applications?

Structure your answer by scan type:

Tool	What It Scans
Trivy	Container images (CVEs in OS packages, libraries)
SonarQube	Source code — bugs, code smells, security hotspots (SAST)
OWASP Dependency Check	Third-party library vulnerabilities

In your pipeline (ApnaKart), Trivy scanned images post-build, SonarQube ran on source code — both as Jenkins stages before deployment.

Q7. What is a vulnerability?

A vulnerability is a weakness or flaw in software, configuration, or infrastructure that can be exploited by an attacker to gain unauthorized access, cause damage, or exfiltrate data.

Examples: unpatched OS packages, exposed secrets in code, misconfigured IAM roles, SQL injection points.

Q8. What vulnerabilities have you encountered in day-to-day work?

"With Trivy, I've seen HIGH and CRITICAL CVEs in base images — mostly from outdated packages in ubuntu:latest or node:18. The fix was switching to minimal base images like alpine or distroless. With SonarQube, common findings were hardcoded credentials flagged as security hotspots, high cognitive complexity in methods, and unused imports flagged as code smells. We addressed blockers before allowing the pipeline to proceed to the next stage."

Q9. What is Cognitive Complexity?

Cognitive Complexity is a SonarQube metric that measures how difficult a piece of code is for a human to understand — not just how many branches it has (that's Cyclomatic Complexity), but how hard it is to mentally trace through.

It penalizes: deeply nested loops/conditionals, multiple breaks/continues, recursion.

High cognitive complexity = harder to maintain, more bug-prone. SonarQube flags methods above a threshold (default: 15).

Q10. What kind of pipelines have you set up in Jenkins?

"I set up a declarative multi-branch pipeline in Jenkins. The stages were:

- Checkout — pull code from GitHub
- SonarQube Analysis — SAST on source code
- Maven Build — compile and package the microservice
- Docker Build — multi-stage Dockerfile, produce image
- Trivy Scan — scan the image for CVEs; pipeline fails if CRITICAL found
- Push to Registry — DockerHub for dev/test, ECR for UAT/prod
- Deploy to EKS — kubectl apply with Kustomize overlays per environment

Branch strategy: dev -> dev namespace, main -> prod namespace. This gave us environment isolation and reduced deployment time by ~60%."

Q11. What is a TF State File in Terraform?

The state file (terraform.tfstate) is Terraform's source of truth — it maps your declared resources to real infrastructure objects in the cloud. Terraform uses it to calculate diffs (what to create, update, destroy) on each plan/apply.

- Never commit it to Git — it can contain secrets

- Store remotely (S3 + DynamoDB for locking) in team environments
- State locking prevents concurrent apply conflicts

In ApnaKart, S3 was used as the remote backend.

Q12. What infrastructure size are you managing?

"In my internship project (ApnaKart), I managed a 10-microservice EKS cluster — 3 worker nodes (t3.medium), with HPA configured per service. Infrastructure included: 1 VPC with public/private subnets across 2 AZs, 1 RDS (PostgreSQL), 1 S3 bucket + CloudFront, ALB, Route53 hosted zone. Total Terraform modules: 6. It was a project-scale infrastructure, not enterprise scale — I want to be upfront about that."

■ *Don't exaggerate. Interviewers probe this.*

Q13. Do you manage client-based or centralized infrastructure?

In your context (internship project), it was centralized — one AWS account, one EKS cluster, multiple namespaces (dev/test/uat/prod) as environment separation.

In enterprise contexts: client-based = separate AWS accounts per client/environment (AWS Organizations + Control Tower). Centralized = shared services model.

Q14. Which deployment strategy do you use for pipelines?

In ApnaKart, Rolling Updates were used on Kubernetes — kubectl applies new pod versions incrementally, old pods terminate as new ones become healthy.

Also mention you understand Blue/Green and Canary as alternatives (see Q15 below).

Q15. What is Blue/Green Deployment?

Two identical production environments run simultaneously:

- Blue = current live environment
- Green = new version deployed and tested

Traffic is switched from Blue to Green via load balancer (ALB weighted target groups, Route53 weighted routing, or Kubernetes service selector swap). If Green has issues, rollback = flip traffic back to Blue instantly.

- Advantage: Zero downtime, instant rollback
- Disadvantage: Double infrastructure cost during transition

In AWS, commonly implemented with: ALB + weighted target groups, Route53 weighted records, AWS CodeDeploy blue/green mode.

Prep Note: Questions 12 and 13 are where interviewers often catch freshers inflating experience. Keep your numbers honest — your ApnaKart metrics (99.9% uptime, 60% deployment time reduction) are strong enough without overstating scale.